```
import sqlite3
from datetime import datetime

# Nombre del archivo físico donde SQLite almacenará la base de datos.
# Si el archivo no existe, SQLite lo creará automáticamente.
DB_FILE = "buenavista.db"


def conectar():
    # Esta función crea y devuelve una conexión con la base de datos SQLite.
    # La conexión será utilizada después para ejecutar consultas SQL.
    conn = sqlite3.connect(DB_FILE)
    return conn


def inicializar_bd(conn):
    # El cursor es el objeto que permite ejecutar sentencias SQL sobre la base de datos.
    cursor = conn.cursor()

    # En SQLite, las claves foráneas no se aplican por defecto.
    # Esta instrucción activa la integridad referencial para que se respeten
    # las relaciones entre tablas y no se generen datos inconsistentes.
    cursor.execute("PRAGMA foreign_keys = ON;")

    # Crear tabla CLIENTES si no existe.
    # Esta tabla almacena la información básica del cliente.
    cursor.execute("CREATE TABLE IF NOT EXISTS clientes (cliente_id INTEGER PRIMARY KEY AUTOINCREMENT, nombre TEXT, apellido

    # Crear tabla AUTOCARAVANAS si no existe.
    # Se usa como clave principal compuesta la matrícula y el país,
    # tal como indica el enunciado.
    # También se relaciona cada autocaravana con su cliente.
    cursor.execute("CREATE TABLE IF NOT EXISTS autocaravanas (matricula TEXT, pais TEXT, cliente_id INTEGER, PRIMARY KEY (ma

    # Crear tabla PARCELAS si no existe.
    # Cada parcela tiene un identificador único y un indicador de actividad.
    cursor.execute("CREATE TABLE IF NOT EXISTS parcelas (parcela_id INTEGER PRIMARY KEY, activa INTEGER);")

    # Crear tabla ESTANCIAS si no existe.
    # Esta tabla registra cada estancia realizada por una autocaravana,
    # incluyendo fechas y estado de la ocupación.
    cursor.execute("CREATE TABLE IF NOT EXISTS estancias (estancia_id INTEGER PRIMARY KEY AUTOINCREMENT, matricula TEXT, pai

    # Crear tabla FACTURAS si no existe.
    # Guarda la información económica generada en el momento del check-out.
    cursor.execute("CREATE TABLE IF NOT EXISTS facturas (factura_id INTEGER PRIMARY KEY AUTOINCREMENT, estancia_id INTEGER,

    # Crear 20 parcelas iniciales si no existen previamente.
    # INSERT OR IGNORE evita error si la parcela ya estaba creada.
    for i in range(1, 21):
        cursor.execute("INSERT OR IGNORE INTO parcelas (parcela_id, activa) VALUES (?, 1);", (i,))

    # Confirmar los cambios realizados en la base de datos.
    conn.commit()


def alta_cliente(conn):
    cursor = conn.cursor()

    # Solicitar por teclado los datos del cliente y de la autocaravana.
    matricula = input("Matrícula: ")
    pais = input("País: ")
    telefono = input("Teléfono: ")
    nombre = input("Nombre: ")
    apellidos = input("Apellidos: ")

    # Insertar primero el cliente en la tabla CLIENTES.
    # Se usan parámetros (?) para evitar inyección SQL y mejorar seguridad.
    cursor.execute("INSERT INTO clientes (nombre, apellidos, telefono) VALUES (?, ?, ?);", (nombre, apellidos, telefono))

    # lastrowid guarda el identificador del último registro insertado.
    # En este caso, se usa para relacionar la autocaravana con el cliente recién creado.
    cliente_id = cursor.lastrowid

    # Insertar la autocaravana en la tabla AUTOCARAVANAS.
    cursor.execute("INSERT INTO autocaravanas (matricula, pais, cliente_id) VALUES (?, ?, ?);", (matricula, pais, cliente_id

    conn.commit()
    print("Cliente registrado correctamente.")
```

```
def ver_parcelas_libres(conn):
    cursor = conn.cursor()

    # Seleccionar las parcelas que no estén ocupadas por una estancia en curso.
    cursor.execute("SELECT parcela_id FROM parcelas WHERE parcela_id NOT IN (SELECT parcela_id FROM estancias WHERE estado='

    # fetchall() recupera todas las filas devueltas por la consulta.
    filas = cursor.fetchall()

    print("Parcelas libres:")
    for fila in filas:
        # Como no usamos sqlite3.Row, el acceso a los datos se hace por posición.
        print(fila[0])


def check_in(conn):
    cursor = conn.cursor()

    # Solicitar los datos necesarios para registrar el ingreso.
    matricula = input("Matrícula: ")
    pais = input("País: ")
    fecha_entrada = input("Fecha entrada (YYYY-MM-DD): ")
    fecha_salida = input("Fecha salida prevista (YYYY-MM-DD): ")

    # Buscar una parcela libre.
    # LIMIT 1 permite seleccionar solo la primera disponible.
    cursor.execute("SELECT parcela_id FROM parcelas WHERE parcela_id NOT IN (SELECT parcela_id FROM estancias WHERE estado='
    fila = cursor.fetchone()

    # Si no hay parcelas disponibles, se informa y se cancela la operación.
    if fila is None:
        print("No hay parcelas disponibles.")
        return

    parcela_id = fila[0]

    # Insertar una nueva estancia en estado EN_CURSO.
    # Esto representa que la autocaravana ha ocupado la parcela.
    cursor.execute("INSERT INTO estancias (matricula, pais, parcela_id, fecha_entrada, fecha_salida_prevista, estado) VALUES

    conn.commit()

    print("Parcela asignada:", parcela_id)


def check_out(conn):
    cursor = conn.cursor()

    # Solicitar los datos necesarios para cerrar la estancia.
    matricula = input("Matrícula: ")
    pais = input("País: ")
    fecha_salida_real = input("Fecha salida real (YYYY-MM-DD): ")

    # Buscar la estancia activa de esa autocaravana.
    cursor.execute("SELECT estancia_id, parcela_id, fecha_entrada, fecha_salida_prevista FROM estancias WHERE matricula=? AN
    fila = cursor.fetchone()

    # Si no existe estancia activa, no se puede generar factura.
    if fila is None:
        print("No hay estancia activa para esa autocaravana.")
        return

    estancia_id = fila[0]
    parcela_id = fila[1]
    fecha_entrada = fila[2]
    fecha_salida_prevista = fila[3]

    # Convertir las fechas de texto a formato datetime para poder operar con ellas.
    fecha1 = datetime.strptime(fecha_entrada, "%Y-%m-%d")
    fecha2 = datetime.strptime(fecha_salida_real, "%Y-%m-%d")

    # Calcular el número de noches de estancia.
    noches = (fecha2 - fecha1).days

    # Validar que la fecha de salida no sea anterior a la de entrada.
    if noches < 0:
        print("Error: la fecha de salida no puede ser anterior a la fecha de entrada.")
        return

    # Calcular el importe total:
    # cargo inicial de 20€ + 40€ por cada noche.
    cargo_inicial = 20
```

```
    tarifa_diaria = 40
    total = cargo_inicial + noches * tarifa_diaria

    # Actualizar la estancia como cerrada.
    cursor.execute("UPDATE estancias SET fecha_salida_real=?, estado='CERRADA' WHERE estancia_id=?;", (fecha_salida_real, es

    # Insertar la factura en la tabla FACTURAS.
    cursor.execute("INSERT INTO facturas (estancia_id, fecha_emision, noches, total) VALUES (?, ?, ?, ?);", (estancia_id, fe

    # Guardar el identificador de la factura recién insertada.
    factura_id = cursor.lastrowid

    conn.commit()

    # Impresión por pantalla de la factura final.
    print("\n==============================")
    print("   CAMPING BUENA VISTA")
    print("        FACTURA")
    print("==============================")
    print("Factura Nº:", factura_id)
    print("Estancia Nº:", estancia_id)
    print("Autocaravana:", matricula, "-", pais)
    print("Parcela:", parcela_id)
    print("------------------------------")
    print("Fecha entrada:", fecha_entrada)
    print("Salida prevista:", fecha_salida_prevista)
    print("Salida real:", fecha_salida_real)
    print("------------------------------")
    print("Noches:", noches)
    print("Cargo inicial:", cargo_inicial, "€")
    print("Tarifa diaria:", tarifa_diaria, "€")
    print("------------------------------")
    print("TOTAL A PAGAR:", total, "€")
    print("==============================")
    print("Pago realizado en el check-out")
    print("==============================\n")


def menu():
    # Menú principal del programa.
    print("\n--- Camping Buena Vista ---")
    print("1) Inicializar BD")
    print("2) Alta cliente")
    print("3) Check-in")
    print("4) Check-out")
    print("5) Ver parcelas libres")
    print("0) Salir")


def main():
    # Crear conexión con la base de datos.
    conn = conectar()

    # Bucle principal del programa.
    while True:
        menu()
        opcion = input("Opción: ")

        if opcion == "1":
            inicializar_bd(conn)

        elif opcion == "2":
            alta_cliente(conn)

        elif opcion == "3":
            check_in(conn)

        elif opcion == "4":
            check_out(conn)

        elif opcion == "5":
            ver_parcelas_libres(conn)

        elif opcion == "0":
            # Salida del programa.
            break

        else:
            print("Opción no válida.")

    # Cerrar siempre la conexión al terminar.
    conn.close()
```

```
if __name__ == "__main__":
    main()
```

```
--- Camping Buena Vista ---
1) Inicializar BD
2) Alta cliente
3) Check-in
4) Check-out
5) Ver parcelas libres
0) Salir
Opción: 1

--- Camping Buena Vista ---
1) Inicializar BD
2) Alta cliente
3) Check-in
4) Check-out
5) Ver parcelas libres
0) Salir
Opción: 2
Matrícula: 00000aaa
País: España
Teléfono: 610000000
Nombre: J
Apellidos: O
Cliente registrado correctamente.

--- Camping Buena Vista ---
1) Inicializar BD
2) Alta cliente
3) Check-in
4) Check-out
5) Ver parcelas libres
0) Salir
Opción: 5
Parcelas libres:
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20

--- Camping Buena Vista ---
1) Inicializar BD
2) Alta cliente
```